

Assignment #3 Analysis Report

Problem 1: Explore the Marabou Resources Directory

- 지원 모델 포맷

Marabou Github Repository의 resources/ 디렉토리에서 세 가지 형식의 샘플 모델이 확인됨

모델 확장자	설명
.nnet	NASA에서 ACAS Xu 항공 충돌 방지 네트워크를 위해 개발한 레거시 형식으로, 완전 연결 네트워크(FC)와 ReLU 활성화 함수만 지원함
.onnx	현재 사실상 표준이 되는 모델 포맷으로, PyTorch 및 TensorFlow 등 대부분의 프레임워크에서 직접 내보낼 수 있으며, Marabou에서 가장 활발하게 지원됨
.pb	TensorFlow Protocol Buffer 형식으로 소형 분류 네트워크이며, .onnx에 비해 유지보수가 적은 편임

- 예제 검증 쿼리

- 저장소의 maraboupy/notebooks/에는 Jupyter 노트북 형태의 사용 예제가 포함되어 있음
- 주요 예제로는 ℓ_∞ -ball 기반 지역 적대적 강건성(local adversarial robustness) 검증 쿼리, 도달 가능성(reachability) 분석 쿼리, 그리고 ACAS Xu 안전 속성(safety property) 명제가 있음
- 입력 명제는 각 입력 차원의 하한·상한 쌍으로 구성된 bound 파일로 제공됨

Problem 2: Run Marabou with an External Model and Dataset

- 설치 과정에서의 호환성 문제 해결

- Marabou는 Github Repository를 프로젝트 폴더 내에 Clone한 후 cmake로 빌드하는 방식으로 가이드에 따라 설치를 진행함
- 빌드 환경은 Ubuntu 24.04, Intel i9-14900KF, Python 3.12(conda 환경)이며, 설치 과정에서 세 가지 주요 오류가 발생되어 아래 표와 같이 원인 규명을 통해 문제를 해결함

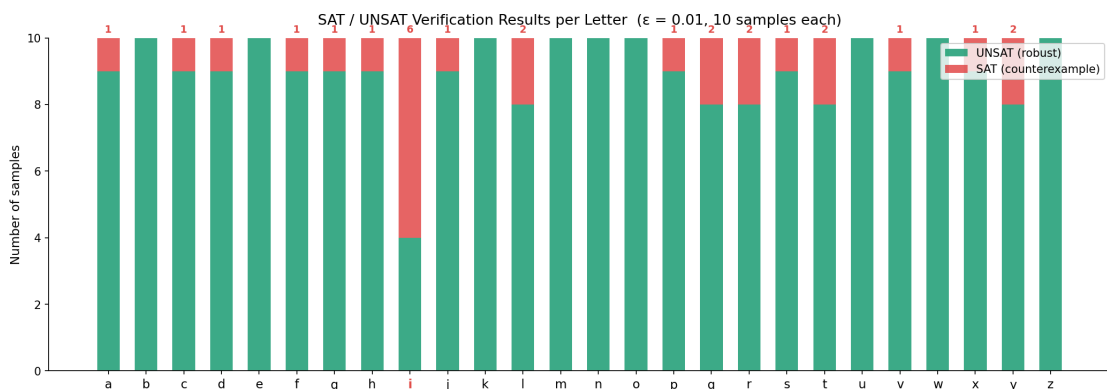
단계	오류	원인	해결
1	<code>libffi LIBFFI_BASE_7.0 undefined symbol</code>	<code>LD_LIBRARY_PATH</code> 의 conda xai 환경 <code>libffi(8.x)</code> 가 시스템 <code>libffi(7.x)</code> 를 덮어씀	<code>env -u LD_LIBRARY_PATH</code> 로 완전 제거
2	OpenBLAS <code>installed/</code> 경로 없음	OpenBLAS가 빌드된 적 없음	수동 빌드 후 install
3	<code>Illegal instruction</code> (테스트 실패)	SKYLAKEX 타겟이 AVX-512 사용 → i9-14900KF는 AVX-512 미지원	<code>TARGET=HASWELL</code> 로 재빌드

- 최종적으로 시스템 OpenBLAS를 직접 지정하여 cmake를 실행함으로써 빌드에 성공함
- 이후 Python 버전 불일치 문제(빌드 시 Python 3.12, 기존 conda 환경 Python 3.11)이 발생하여 Python 3.12로 재설치하여 호환성 문제를 해결함

- 모델 및 데이터 선택

- 데이터: EMNIST Letters

- 26개의 알파벳 클래스(a-z)에 대한 28x28 그레이스케일 손글씨 이미지로 MNIST와 유사함
 - 완전 소형 신경망 구조를 사용하는 것을 고려하여 구조가 단순하고 비교적 가벼운 데이터셋을 활용함
- 모델: FC & ReLU
 - 구조: 784(28X28) → FC(256) → ReLU → FC(128) → ReLU → FC(26)
 - Marabou의 ReLU 기반 검증 엔진과 호환되고 적절하게 소형 규모의 크기를 가짐(약 23만 파라미터)
 - 모델은 Adam Optimizer로 10 Epoch 학습되어 테스트 정확도 90.64% 달성하였으며, ONNX 형식으로 Export함
- 검증 쿼리 정의
 - 입력 이미지 x 가 알파벳 c 로 분류될 때 $l_\infty - ball$ 반경 ϵ 내의 모든 입력 x' 도 동일하게 c 로 분류되는지 검증함
 - 구체적으로, 각 경쟁 클래스 $j \neq c$ 에 대해 별도의 Marabou 쿼리를 실행하여 $output[j] \geq output[c]$ 를 만족하는 입력이 존재하는지 탐색함
 - 어느 하나라도 SAT이면 counterexample이 존재하고, 모두 UNSAT이면 속성이 증명됨
- 실험 구성
 - 실험1: 알파벳당 10개 샘플(총 260개)에 대해 $\epsilon = 0.01$ 로 클래스별 강건성을 평가함
 - 실험2: 시각적으로 유사한 알파벳 쌍(a/w, i/l, o/q 등) 사이를 선형 보간하여 결정 경계 근처의 샘플을 생성하고, 동일한 $\epsilon = 0.01$ 로 SAT 반례를 시연함
- 실험 결과
 - 실험1



- 전체 260개 샘플 중 30개(11.5%)에서 SAT이 확인되었으며, 230개 (88.5%)는 $\epsilon = 0.01$ 범위 내에서 강건성이 형식적으로 증명됨
 - 알파벳 'i'는 60%의 SAT 비율로 가장 취약한 클래스로 확인되었으며, 이는 'i'가 'j', 'l' 등과 시각적으로 유사하여 결정 경계가 좁아 발생한 것으로 예상됨
 - 반면 [b, e, k, m, n, o, u, w, z] 샘플에 대해서는 10개 샘플 모두 UNSAT으로 완전한 강건성이 증명됨
 - 검증 시간은 SAT 케이스에서 0.5~4초, UNSAT 케이스에서 3~6초가 일반적이었으나, 일부 샘플에서 비정상적으로 긴 탐색 시간이 확인됨
- 실험2

알파벳 쌍	결과	반례 클래스	검증 시간	비고
a ↔ w	SAT	w	8.71s	
i ↔ l	SAT	l	3.02s	
o ↔ q	SAT	a	0.53s	예상 외 클래스로 분류
c ↔ e	SAT	e	1.19s	
n ↔ i	SAT	m	3.17s	예상 외 클래스로 분류
u ↔ v	-	-	-	경계 샘플 생성 실패

- 실험을 수행한 5쌍 중 5쌍 모두 반례가 확인되었으며, 이 중 $o \leftrightarrow q$, $n \leftrightarrow j$ 쌍에서 예상과는 전혀 다른 클래스 각 a와 m으로 분류하는 반례가 발견됨
- 이는 결정 경계 근처에서 여러 클래스가 인접하는 복잡한 분류 경계가 존재함을 시사함
- $u \leftrightarrow v$ 쌍은 선형 보간으로 생성된 중간 샘플이 모두 u로 분류되어 경계 샘플 생성에 실패하였으며, 이는 두 클래스 간 결정 경계가 비선형적임을 나타냄
- Marabou의 장단점
 - 장점
 - Completeness와 Soundness를 보장하는 형식 검증 도구로, UNSAT 결과는 통계적 추정이 아닌 수학적 증명으로, 일정 범위 내에 모든 입력에 대한 강건성을 보장함
 - Python API를 통해 ONNX 모델 로드와 입출력 제약 조건 설정이 직관적이며, 소형 네트워크와 작은 ϵ 값에서 검증이 수 초 이내에 비교적 빠르게 수행됨
 - 단점
 - branch-and-bound 기반 탐색은 ReLU 활성화 패턴 수에 지수적으로 의존하므로, ϵ 이 커지거나 네트워크가 깊어질수록 검증 시간이 급격히 증가함
 - 지원 연산자가 제한적이어서 BatchNorm, Softmax 등을 포함한 모델은 별도의 구조 수정이 필요함